

Evaluation of removing the GIL from a multi-threaded Python Trading Simulation

Louis Tanak, Student at The University of Warwick

September 27, 2025

1 Introduction

Recently, the Python 3.14 pre-release was unveiled, with my interest being drawn to the point PEP 779: Free-threaded Python is officially supported.

A while ago I created a trading simulation, which utilised the `threading` library to concurrently process multiple stocks and handle API requests for a UI. The multi-threading provided a significant performance improvement due to my IO-based implementation (please see 2.2). However, with the recent Python news, I decided to further investigate the result of removing the GIL and its impact on my code, but also its impact on heavy CPU-bound threads.

2 How the Trading Simulation works

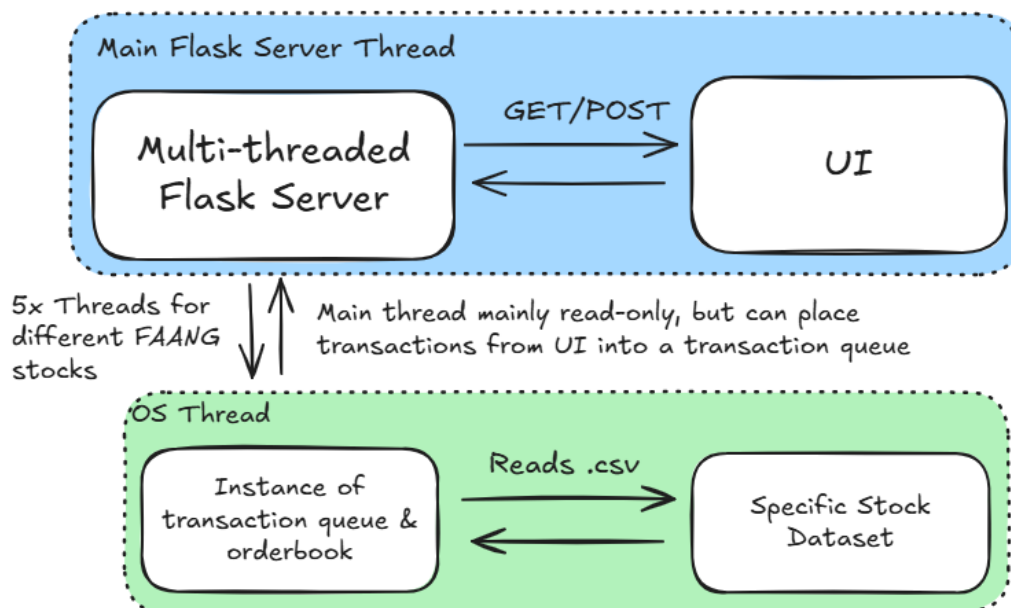
2.1 Project Overview

The simulation was a project I created to understand how financial exchanges work. I wanted to simulate other users, as well as enable the user to submit their own trades. Orders would get matched via a price-time priority algorithm, then be displayed to the user via a UI.

As the project was to just understand how exchanges work, to simulate other traders I used datasets of buy / sell orders of various FAANG stocks. This data is stored in `.csv` files, and we read the transactions line by line to form our order queue.

Below is a diagram of how the project works.

Figure 1: Diagram showing the generalised project



2.2 Use of multi-threading

Originally when I implemented the multi-threading, each thread would pre-load the data and then perform its execution. This worked, but is incredibly memory inefficient and there was clear performance issues. The reason behind this is due to how the Global Interpreter Lock (GIL) works in Python.

The GIL is a mutex lock that ensures only one thread can execute Python bytecode in the interpreter. It was created like this due to the nature of Python's garbage collector that uses a reference counting based system, which without the GIL the reference counts would be corrupted in multi-threaded programs.

Because of this, it means if you are utilising CPU-bound threads, you will have an insignificant speed up, as only one thread can execute at a time. This means if you were to use preloading, it leaves you with CPU-bound threads, which are bottlenecked by the GIL.

However, by reading directly from the CSV it provides a performance improvement. I/O operations in Python release the GIL while the program waits for the disk to respond. This means that whilst one thread is blocked on I/O and reading its next data point, another thread can execute its Python code by acquiring the GIL. The result of this is that the program overlaps I/O waits with CPU computation, achieving concurrency and a higher throughput compared to the previous approach.

It is also less memory intensive and more realistic to how a real exchange works (orders coming in one by one). The summary of this is file reading has I/O waits, but those waits allow Python to schedule other threads, providing better throughput under the GIL.

With both of the approaches now defined, we can move onto the testing.

3 Testing

The tests that will be performed are the following:

- The effect of free-threaded Python on the current implementation - Expecting minimal improvements.
- The effect of free-threaded Python on the pre-loading / CPU-based implementation - Expecting significant improvements.

With the current implementation, I believe removing the GIL will have a small effect on the execution time of the program. This is because the majority of the runtime will now be waiting on the CSV reads, and the GIL had minimal contention in the original code.

However, using the pre-loading approach there should be a significant improvement as its original bottleneck was the GIL. Without it, we should see a major improvement and potentially its implementation being faster than the I/O based solution, though it will still possess the downside of it being memory inefficient.

3.1 Benchmarking Library

To perform the tests, I created a simple benchmarking library. It implements the ability to run the simulation on multiple runs, then outputs the mean computation time and the standard deviation between the runs specified. It can also set a limit to the amount of orders it processes, so we can test against the scaling of the data.

3.2 Base results

As Python3.14 as of writing this is currently in pre-release, I will be conducting my tests using the experimental version of free-threaded Python in version 3.13.7.

Below are the base table of results for the I/O and the CPU-based implementations using Python3.13.

Table 1: I/O Based Implementation

Order Limit	Runs	Mean Time (s)	Median Time (s)	Std. Deviation
50000	3	0.6344	0.6150	0.0579
50000	10	0.6069	0.5993	0.0254
200000	3	1.5518	1.5561	0.0082
200000	10	1.6725	1.6724	0.0564
Unlimited (411409)	3	2.7254	2.7231	0.0347
Unlimited (411409)	10	2.9670	2.9174	0.1253

Table 2: CPU Based Implementation

Order Limit	Runs	Mean Time (s)	Median Time (s)	Std. Deviation
50000	3	1.4977	1.4940	0.0535
50000	10	1.5437	1.5355	0.0572
200000	3	2.5269	2.5033	0.0880
200000	10	2.4717	2.4932	0.0973
Unlimited (411409)	3	3.5937	3.4676	0.3190
Unlimited (411409)	10	3.8732	3.8550	0.3795

3.3 Free-threaded I/O Implementation

Below are the results from running the I/O based approach with Python3.13.7t.

Table 3: I/O Based Implementation - Free-threaded

Order Limit	Runs	Mean Time (s)	Median Time (s)	Std. Deviation	Mean Improvement (%)
50000	3	0.2605	0.2571	0.0078	+58.938%
50000	10	0.2516	0.2495	0.0057	+58.543%
200000	3	0.9694	0.9730	0.0066	+37.531%
200000	10	0.9652	0.9557	0.0304	+42.290%
Unlimited (411409)	3	1.9512	1.9440	0.0271	+28.407%
Unlimited (411409)	10	1.9753	1.9693	0.0236	+33.424%

These performance increases are significant improvements, considering the original implementation was an efficient approach in previous Python versions. As the execution is now truly parallel, we can see that we have reduced the context-switch overhead of the threads.

The next set of results will also be able to help indicate the impact of using an I/O to pre-load the data.

3.4 CPU Based Implementation - Free-threaded

Below are the results from running the CPU-based implementation with the free-threaded Python version.

Table 4: CPU Based Implementation - Free-threaded

Order Limit	Runs	Mean Time (s)	Median Time (s)	Std. Deviation	Mean Improvement (%)
50000	3	0.5682	0.4634	0.1902	+62.062%
50000	10	0.4954	0.4659	0.1004	+67.908%
200000	3	1.2098	1.1523	0.1705	+52.123%
200000	10	1.1143	1.0818	0.0827	+54.918%
Unlimited (411409)	3	2.0300	1.9727	0.1451	+43.512%
Unlimited (411409)	10	2.0562	2.0550	0.0941	+46.912%

These results are incredibly interesting. We achieved the expected result of a significant speedup in the approach, however for lower order limits we can see that it performs worse than the I/O based approach, but as we increase the order limit we tend towards the same execution time.

For smaller work-loads, the reason for the CPU-based approach for being slower could be down to cache contention between the threads. As we have pre-loaded the data, each thread will want their data into the cache so that it could be retrieved quickly. However, the OS scheduler may put each of the threads on the same cores, therefore each thread will be competing over the same cache lines, causing the contention. This effect is reduced at the higher order limits, due to the larger datasets spreading over a

larger address range. As we have more data across more addresses, not all of it will fit in the cache, reducing the probability of threads working on the same cache lines and more likely retrieving data from main memory. At this point, we would now face cache capacity as our bottleneck instead of cache contention.

4 Conclusion

In conclusion, we can see a clear improvement in multi-threaded performance by removing the GIL. This improvement could impact thousands of multi-threaded Python codebases, especially simplistic multi-threaded servers ranging from personal projects like this one to larger scale systems.

Thank you for reading, I thoroughly enjoyed having a deeper dive into one of my old projects.

GitHub repository: [MatchingEngine](#)